Università di Genova

UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

MODELLING AND CONTROL OF MANIPULATORS

Second Assignment

Manipulator Geometry and Direct Kinematics

Author:

Spluga Matteo
Nogarin Giacomo
Tozzola Michele

Student ID:

s9185902
s8654515
s8995647

Professors:

Enrico Simetti
Giorgio Cannata

Tutors:

Simone Borelli
Luca Tarasi

December 17, 2025

Contents

1 Assignment description **3**
1.1 Exercise 1 . . . 3

2 Exercise 1 **5**
2.1 Q1.1 . . . 5
2.2 Q1.2 . . . 6
2.3 Q1.3 . . . 8
2.4 Q1.4 . . . 8
2.5 Q1.5 . . . 9
2.6 Q1.6 . . . 10
2.7 Q1.7 . . . 10

Mathematical expression	Definition	MATLAB expression
$< w >$	World Coordinate Frame	w
${}^{a}_{b}R$	Rotation matrix of frame $< b >$ with respect to frame $< a >$	aRb
${}^{a}_{b}T$	Transformation matrix of frame $< b >$ with respect to frame $< a >$	aTb

Table 1: Nomenclature Table

1 Assignment description

The second assignment of Modelling and Control of Manipulators focuses on manipulators' geometry and direct kinematics.

- Download the .zip file from the Aulaweb page of this course.
- Implement the code to solve the exercises on MATLAB by filling the template classes called *geometricModel* and *kinematicModel*
- Write a report motivating your answers, following the predefined format on this document.

1.1 Exercise 1

Given the following CAD model of an industrial 7 DoF manipulator:

Q1.1 Define all the model transformation matrices, by filling the structures in the *BuildTree()* function. Be careful to define the z-axis coinciding with the joint rotation axis, such that the positive rotation is the same as showed in the CAD model you received; and the x-axis, when possible, pointing to the next link. Draw on the CAD model the reference frames for each link and insert it into the report.

Q1.2 Implement the method of *geometricModel* called *updateDirectGeometry()* which computes the model transformation matrices as a function of the joint position q. Explain the method used and comment on the results obtained.

Q1.3 Implement the method of *geometricModel* called *getTransformWrtBase()* which computes the transformation matrix from the base to a given frame. Using this method, compute the following transformation matrices: ${}^{b}_{e}T$, ${}^{6}_{2}T$. Explain the method used and comment on the results obtained.

Q1.4 Run the MATLAB section Q1.4 in the given script *main.m*, which computes the configurations of the manipulators moving from an initial joint position $q_i = [\frac{\pi}{4}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.15, \frac{\pi}{4}]^\top$ to $q_f = [\frac{5\pi}{12}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.18, \frac{\pi}{5}]^\top$. The script generates plots using methods from the given class *plotManipulators*, which must not be modified. Check that it behaves reasonably and show the obtained plots in the report.

Q1.5 Implement the method of *kinematicModel* called *getJacobianOfLinkWrtBase()* which takes as argument the link number, and returns the Jacobian of the corresponding link with respect to the base. Compute the Jacobian of **link 6** with respect to base for the final configuration q_f given in Q1.4. Explain the method used and comment on the result obtained.

Q1.6 Implement the method of *kinematicModel* called *updateJacobian()* which updates the Jacobian matrix of the manipulator for the end-effector. Compute the Jacobian for the final configuration q_f given in Q1.4. Explain the method used and comment on the results obtained.

Q1.7 Given the following joint position, $q = [0.7, -0.1, 1, -1, 0, 0.03, 1.3]^\top$ expressed in radians and meters, and the following joint velocities $\dot{q} = [0.9, 0.1, -0.2, 0.3, -0.8, 0.5, 0]^\top$ expressed in rad/s and m/s, compute the velocities of the end effector with respect to the base projected on the end effector frame, ${}^{e}v_{e/b}$ and ${}^{e}\omega_{e/b}$, using **forward kinematics**.

Remark: All the methods must be implemented for a generic serial manipulator. For instance, joint types, and the number of joints should be parameters.

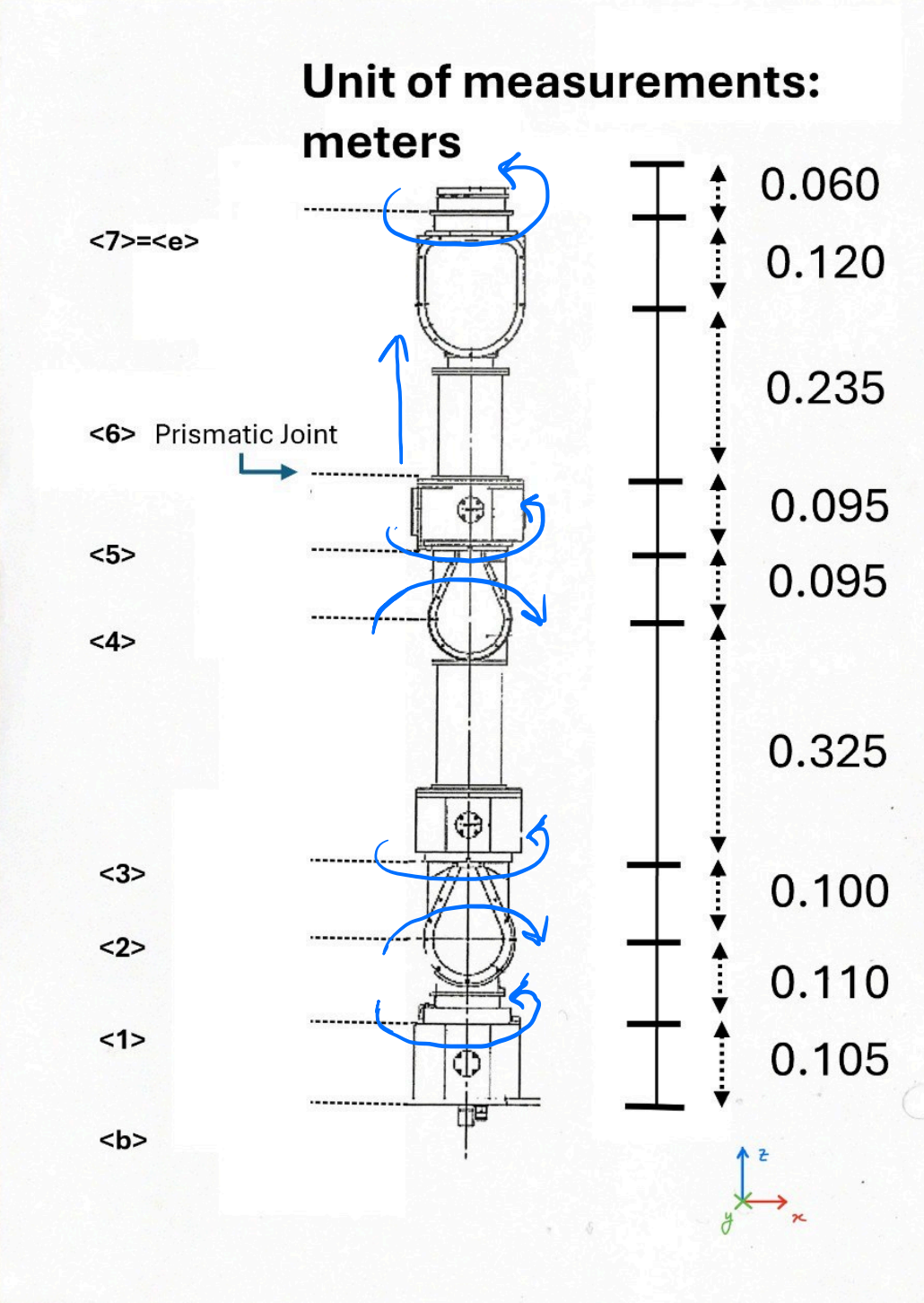


Figure 1: CAD model of the robot. The directions of motion of each joint are indicated by the blue arrows.

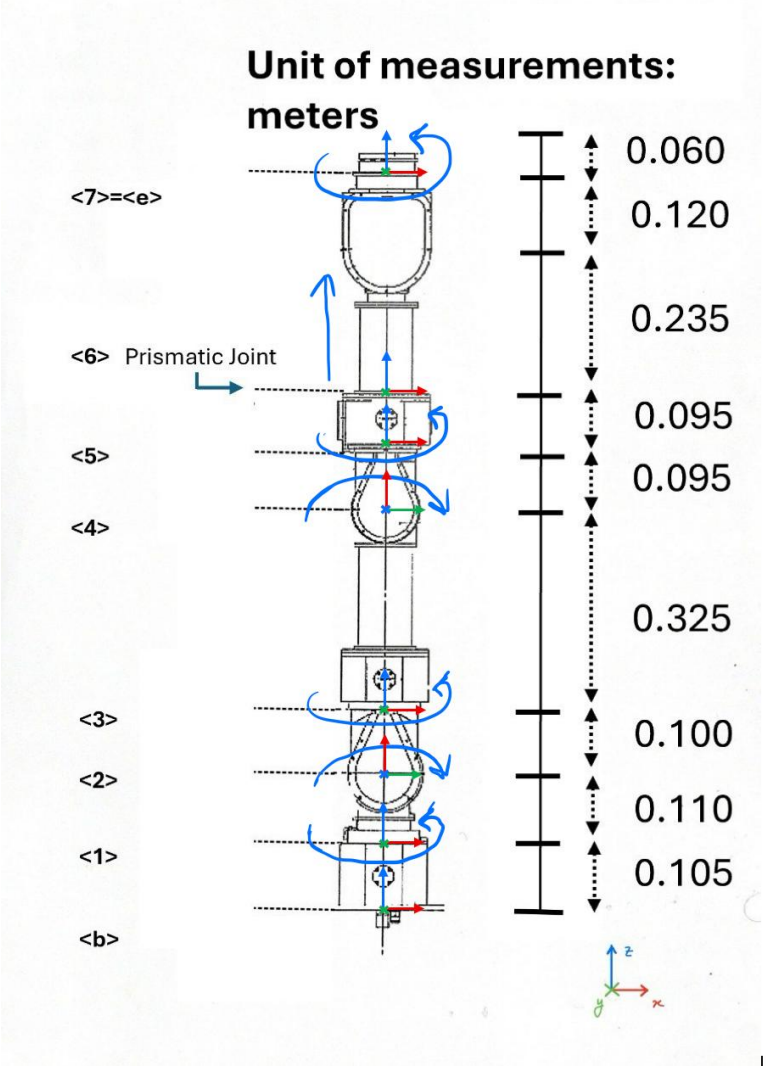
2 Exercise 1

2.1 Q1.1

Given the CAD model of a 7 DoF robot shown in Fig. 1 considered for this assignment, the starting point of its direct kinematic analysis regards the definition of the transformation matrices of the model. This was done by filling the *BuildTree()* function.

In the CAD model of the manipulator, the reference frames were placed with the z-axes aligned with the rotational axes for rotational joints and along the displacement axes for the prismatic revolut. When it was possible, the x-axes were drawn pointing towards the next link.

The following is the CAD model of the robot including the frames oriented for each joint.



The resulting transformation matrices regarding the definition of the frames in the model are as follows:

$$ {}^{b}_{1}T = \begin{bmatrix} 1.0000 & 0 & 0 & 0 \\ 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 1.0000 & 0.1050 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} $$

$$ {}^{1}_{2}T = \begin{bmatrix} 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 1.0000 & 0 \\ 1.0000 & 0 & 0 & 0.1100 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} $$

$$ {}^{2}_{3}T = \begin{bmatrix} 0 & 0 & 1.0000 & 0.1000 \\ 1.0000 & 0 & 0 & 0 \\ 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} $$

$$ {}^{3}_{4}T = \begin{bmatrix} 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 1.0000 & 0 \\ 1.0000 & 0 & 0 & 0.3250 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} $$

$$ {}^{4}_{5}T = \begin{bmatrix} 0 & 0 & 1.0000 & 0.0950 \\ 1.0000 & 0 & 0 & 0 \\ 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} $$

$$ {}^{5}_{6}T = \begin{bmatrix} 1.0000 & 0 & 0 & 0 \\ 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 1.0000 & 0.0950 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} $$

$$ {}^{6}_{7}T = \begin{bmatrix} 1.0000 & 0 & 0 & 0 \\ 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 1.0000 & 0.3550 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} $$

It can be observed that, whenever the direction of the axis did not change, the rotation matrices inside the transformation ones is an identity matrix.

2.2 Q1.2

The transformation matrices of the model have also been computed taking into account the joint configurations *q*. In the previous circumstance, the matrices were built considering $q = 0$, so that they could represent the transformations between the frame of one joint and the next considering their fixed geometry. In this case, the transformation matrices were calculated (based on their fixed geometry configuration) by adding their movement effect (considered with $q \neq 0$), which describes the real pose of the joints. In practice, whenever a rotational joint (rotating about the z-axis) was met, the following resulting matrix was obtained:

$$ {}^{i}_{i+1}T(q_i) = {}^{i}_{i+1}T(0)T_{\text{joint}}(q_i) = {}^{i}_{i+1}T(0) \begin{bmatrix} \cos q_i & -\sin q_i & 0 & 0 \\ \sin q_i & \cos q_i & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

For the prismatic joint, the following resulting matrix was obtained instead:

$$ {}^{i}_{i+1}T(q_i) = {}^{i}_{i+1}T(0)T_{\text{joint}}(q_i) = {}^{i}_{i+1}T(0) \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & q_i \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

This computation was done using the *updateDirectGeomery()* function which utilizes parameters from the *geometricModel* constructor belonging to the *geometricModel* class.

For the joint configuration indicated in Q1.4 $q = [\pi/4,\ -\pi/4,\ 0,\ -\pi/4,\ 0,\ 0.15,\ \pi/4]$, the method *updateDirectGeometry()* produces the local transformation matrices ${}^{i}_{i+1}T$ reported below. These matrices describe the relative pose between consecutive joint frames and clearly show the effect of the revolute and prismatic joints on the rotational and translational components, respectively. The resulting matrices computed with the previously described MATLAB function are the following:

$${}^{0}_{1}T(q_1) = \begin{bmatrix} 0.7071 & -0.7071 & 0 & 0 \\ 0.7071 & 0.7071 & 0 & 0 \\ 0 & 0 & 1.0000 & 0.1050 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

The rotation of 45° about the z-axis indicates that the first joint frame is not aligned with the base frame.

$${}^{1}_{2}T(q_2) = \begin{bmatrix} -0.7071 & 0.7071 & 0 & 0 \\ 0 & 0 & 1.0000 & 0 \\ 0.7071 & 0.7071 & 0 & 0.1100 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

The rotational component realigns the reference axes to accommodate the kinematic structure of the following joint.

$${}^{2}_{3}T(q_3) = \begin{bmatrix} 0 & 0 & 1.0000 & 0.1000 \\ 1.0000 & 0 & 0 & 0 \\ 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

The matrix ${}^{2}_{3}T$ is the same of $T(0)$ because the q of joint three is zero.

$${}^{3}_{4}T(q_4) = \begin{bmatrix} -0.7071 & 0.7071 & 0 & 0 \\ 0 & 0 & 1.0000 & 0 \\ 0.7071 & 0.7071 & 0 & 0.3250 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

The transformation ${}^{3}_{4}T$ describes a rotation of 45° about the z-axis.

$${}^{4}_{5}T(q_5) = \begin{bmatrix} 0 & 0 & 1.0000 & 0.0950 \\ 1.0000 & 0 & 0 & 0 \\ 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

The matrix ${}^{4}_{5}T$ introduces a reorientation of the reference frame and a short translation.

$${}^{5}_{6}T(q_6) = \begin{bmatrix} 1.0000 & 0 & 0 & 0 \\ 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 1.0000 & 0.2450 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

The transformation ${}^{5}_{6}T$ describes the prismatic joint of the manipulator. Since no rotation is involved, the orientation remains unchanged.

$${}^{6}_{7}T(q_7) = \begin{bmatrix} 0.7071 & -0.7071 & 0 & 0 \\ 0.7071 & 0.7071 & 0 & 0 \\ 0 & 0 & 1.0000 & 0.3550 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

The transformation ${}^{6}_{7}T$ defines the pose of the end-effector frame with respect to the last joint frame. The rotational component sets the final orientation of the tool, whereas the translation accounts for the terminal link length of the manipulator.

From the obtained transformation matrices, ìo5it can be observed that each local transformation ${}^{i}_{i+1}T$ correctly describes both the relative orientation and the geometric offset between consecutive joint frames. Every translational component is consistent with the dimensions reported in the CAD model, while the rotational terms account for the non-aligned joint axes and the kinematic structure of the manipulator.

2.3 Q1.3

The implementation of the method of *geometricModel* called *getTransformWrtBase()* was then implemented. It computes the transformation matrix from the base to a given frame. In this case, this method was used to compute the transformation matrix between the end effector ”e” and the base joint ”b”. Later, the transformation matrix between joints 2 and 6 was also calculated. (Note: we are considering the home position, i.e $q = 0$)

The resulting matrix from the base joint to the end effector is the following:

$$
{}^{b}_{e}T = \begin{bmatrix} 1.0000 & 0 & 0 & 0 \\ 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 1.0000 & 1.1850 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}
$$

The method *getTransformWrtBase(k)* computes the homogeneous transformation matrix ${}^{0}_{k}T$ by chaining the local transformations between consecutive joint frames. Starting from the identity matrix, each local transform ${}^{i-1}_{i}T$ is successively multiplied, resulting in the pose of the k-th joint frame with respect to the base frame.

The obtained transformation matrix represents a pure translation along the z axis, as the rotational part is the identity matrix. This indicates that there is no relative rotation between the considered frames, while the positional difference is entirely due to a vertical displacement. The translation value of $1.185\,\mathrm{m}$ is therefore consistent with the cumulative contribution of the fixed link lengths and the extension of the prismatic joint in the considered configuration shown in Fig. 1.

The resulting matrix from the joint 2 to joint 6 is the following:

$$
{}^{6}_{2}T = \begin{bmatrix} 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 1.0000 & 0 \\ 1.0000 & 0 & 0 & -0.6150 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}
$$

The chosen computation method for this matrix is done by writing another function in the *geometricModel* class called *getTransformWrtRandomFrame()*. This takes the first joint (6) and the second one (2) as inputs and performs the transformation with respect to the base by calling the previously defined function *getTransformWrtBase()*, giving ${}^{0}_{6}T$ and ${}^{0}_{2}T$ as results. ${}^{6}_{2}T$ is then calculated by multiplying the inverted ${}^{0}_{6}T$ matrix (which will be ${}^{6}_{0}T$) to the ${}^{0}_{2}T$ matrix.

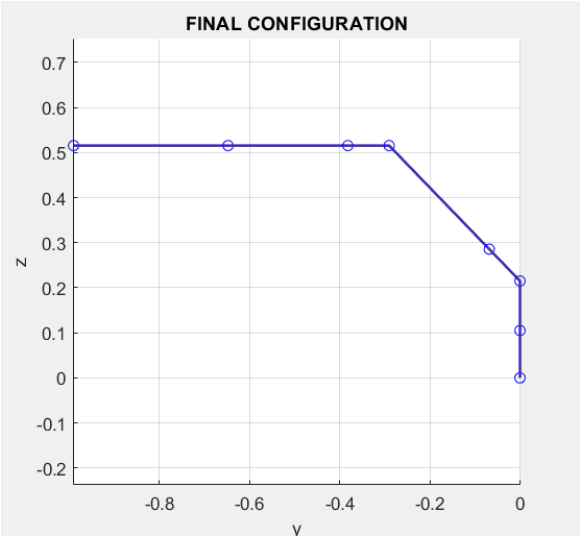
2.4 Q1.4

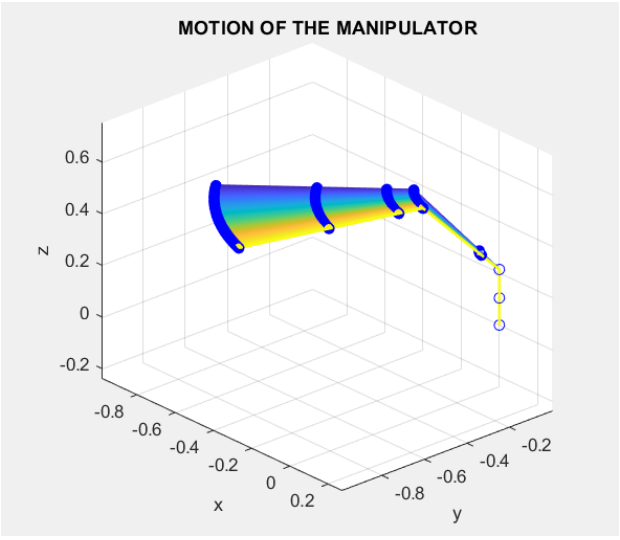
In this subsection the resulting movement simulation of the manipulator between two joint positions q_i and q_f is shown. The respective positions are described here more in detail:

$\mathbf{q}_i = \left[\frac{\pi}{4}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.15, \frac{\pi}{4}\right]$

$\mathbf{q}_f = \left[\frac{5\pi}{12}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.18, \frac{\pi}{5}\right]$

The movement plot, with also the final configuration of the manipulator, computed with the use of methods contained in the *plotManipulators* class, is subsequently shown:





From the reported figures it can be observed that the manipulator reaches a physically consistent final configuration corresponding to the specified joint values. The end-effector position and orientation are coherent with the kinematic structure of the robot and with the contribution of both revolute and prismatic joints. Moreover, the simulated motion shows a smooth and continuous trajectory, confirming that the local transformation matrices are correctly updated at each time step and that the direct kinematics is properly computed throughout the entire motion.

2.5 Q1.5

The implementation of the method of *kinematicModel* called *getJacobianOfLinkWrtBase()*, which takes the link number as input, computes the Jacobian of that link with respect to the base frame. Considering the final configuration q_f indicated in Q1.4, this method was then used to calculate the Jacobian of link 6 with respect to the base. The Jacobian is built column by column by considering only the joints that affect the motion of the selected link. In fact, each column represents the velocity effects (angular and linear) that each link has on the sixth link.

The base form of the Jacobian matrix is the following:

$$\mathbf{J}(\mathbf{q}) = \left[\begin{array}{c} \mathbf{J}_\omega(\mathbf{q}) \\ \hline \mathbf{J}_v(\mathbf{q}) \end{array}\right]$$

For revolute joints, each column accounts for both angular and linear velocity contributions:

$$\mathbf{J}_i = \begin{bmatrix} \mathbf{z}_i \\ \mathbf{z}_i \times (\mathbf{p}_j - \mathbf{p}_i) \end{bmatrix}$$

For prismatic joints only the linear contribution is present:

$$\mathbf{J}_i = \begin{bmatrix} \mathbf{0} \\ \mathbf{z}_i \end{bmatrix}$$

where z_i is the axis the j joint rotates about. p_j is the distance of frame j and p_i is the position of the i frame both with respect to the base frame.

As a result, the Jacobian has size $6 \times j$ and correctly relates joint velocities to the spatial velocity of link j.

The resulting Jacobian matrix computed for joint 6 can be observed here:

$${}^0_6\mathbf{J} = \left[\begin{array}{cccccc} 0 & -0.9659 & -0.1830 & -0.9659 & -0.2588 & 0 \\ 0 & 0.2588 & -0.6830 & 0.2588 & -0.9659 & 0 \\ 1.0000 & 0 & 0.7071 & 0 & 0 & 0 \\ \hline 0.6477 & 0.0778 & 0.2527 & 0 & 0 & -0.2588 \\ -0.1735 & 0.2903 & -0.0677 & 0 & 0 & -0.9659 \\ 0 & 0.6705 & 0 & 0.3700 & 0 & 0 \end{array}\right]$$

From the obtained matrix, different considerations can be taken into account. Firstly, on the sixth column, it can be clearly seen that the sixth joint is prismatic (no angular values). Looking at the last three rows of the sixth column, it is possible to see that the linear velocity has non zero components only for the x and y components, therefore the joint 6 moves link 6 along a direction that lays on the xy plane. This shows that link 6 is inclined and not vertical unlike it is shown in the CAD model. This suggests that the computation in the *getJacobianOfLinkWrtBase()* method was done correctly, considering the final configuration q_f. Then, the first three columns show that the first three links give a great contribute to the motion and orientation of link 6, while links 4 and 5 only produce local rotations while almost not giving a translational contribute. When also looking to the fifth column, it can be seen that no linear values are different from zero. This happens because of the alignment of link 5 with the end-effector such that no linear velocities impose an effect on the end-effector movement.

2.6 Q1.6

Here it is shown the implementation of the method *updateJacobian()* from *kinematicModel* which updates the Jacobian matrix of the manipulator for the end-effector starting from the geometric Jacobian of the last link. The angular and linear components are first separated, and the linear part is then updated to account for a possible offset between the end-effector and the final link frame. This correction is performed using the cross-product operator associated with the offset vector. In the considered case, the offset is zero, so the resulting Jacobian coincides with that of the final link.

The resulting matrix is the following:

$$
{}^{0}\mathbf{J}_{e} = \left[\begin{array}{ccccccc}
0 & -0.9659 & -0.1830 & -0.9659 & -0.2588 & 0 & -0.2588 \\
0 & 0.2588 & -0.6830 & 0.2588 & -0.9659 & 0 & -0.9659 \\
1.0000 & 0 & 0.7071 & 0 & 0 & 0 & 0 \\
\hline
0.9906 & 0.0778 & 0.4952 & 0 & 0 & -0.2588 & 0 \\
-0.2654 & 0.2903 & -0.1327 & 0 & 0 & -0.9659 & 0 \\
0 & 1.0255 & 0 & 0.7250 & 0 & 0 & 0
\end{array}\right]
$$

The seventh column of the Jacobian corresponds to the last revolute joint of the manipulator. From the matrix, it can be observed that the linear component associated with this joint is zero, while the angular contribution is applied only on the x and y coordinates, therefore its axis lays on this plane. This indicates that, for the current pose, the seventh joint mainly affects the local orientation of the tool without producing a significant translational motion of the end-effector. Therefore, its effect on the end-effector position is negligible in this posture.

2.7 Q1.7

In this last part of the assignment the angular and linear velocities of the end-effector (with respect to the base projected on the end-effector frame) were computed by using the forward kinematics method. This was done considering two vectors which contained the joint positions and velocities. The vectors are subsequently shown:

$$\mathbf{q} = \begin{bmatrix} 0.7 & -0.1 & 1 & -1 & 0 & 0.03 & 1.3 \end{bmatrix}^{\top}$$

$$\dot{\mathbf{q}} = \begin{bmatrix} 0.9 & 0.1 & 0.2 & 0.3 & 0.8 & 0.5 & 0 \end{bmatrix}^{\top}$$

Given these new values, the methods *updateDirectGeometry(q)* and *updateJacobian()* have been called to update respectively the configuration and the jacobian of the robot. Subsequently, the velocities of the end effector were computed through:

$$
{}^{b}\nu_{ee} = \begin{bmatrix} {}^{b}\omega_{ee/b} \\ {}^{b}v_{ee/b} \end{bmatrix} = J_{ee}\dot{q}
$$

After this computation, since now the velocity is expressed with respect to the base frame, the final step consists of expressing the velocities from the end effector frame.

${}^{ee}\omega_{ee/b} = {}^{b}R_{ee}^{T} * {}^{b}\omega_{ee/b}$

${}^{ee}v_{ee/b} = {}^{b}R_{ee}^{T}({}^{b}v_{ee/b} - [r\times]{}^{b}\omega_{ee/b})$